```
"""Shared camera and bullet helpers.

This module exists to stop dinosaur.py and dinosaurs.py from each rolling their
own per-frame camera, shake-update, bullet/wall collision, and bullet render
code. The two dinosaurs had drifted on every one of these axes:

  - Camera lerp factor (0.05 vs 0.10 / 0.08 / smoothstep)
  - Camera first-frame snap (D1 lerps from (0,0); D2 snaps)
  - Camera map-bounds clamp (D1 had none; D2 clamps)
  - Camera recoil-kick (D1 dropped it; D2 applied it)
  - Bullet/wall collision (D1 point-sample WORLD; D2 swept clipline)
  - Bullet render (D1 had glow halo; D2 did not)

All of these now live here. Both dinosaurs call ``update_camera`` once per
frame and ``draw_bullets`` once per render.
"""
import math
import random
import pygame

from config import WIDTH, HEIGHT, TILE_SIZE, C_BULLET, bullet_glow
from state import STATE

def update_camera(target, map_w=None, map_h=None, focus_factor=None,
                  handheld_jitter_time=None, factor_fn=None):
    """Per-frame camera update with shake, recoil kick, lerp, and clamp.

    Args:
        target: (x, y) world position the camera should follow (usually the
            player's rect center, possibly adjusted for cutscene focus).
        map_w, map_h: Optional map dimensions in tiles. If provided, the
            "desired" camera position is clamped to ``[0, map_w*TILE_SIZE - WIDTH]``
            and the same for y BEFORE the lerp. Pass these to prevent the camera
            from running past the map edge into black void.
        focus_factor: Optional constant lerp override (0..1) for cutscene
            focus pulls. None means use the default ease (0.10).
        factor_fn: Optional callable ``(dx, dy, dist) -> float`` that computes
            a per-frame lerp factor from the offset to the target. Used by D2's
            output stage for distance-scaled smoothstep easing. Overrides
            ``focus_factor`` if provided.
        handheld_jitter_time: Optional int. When not-None and not-zero, adds
            a subtle handheld sway using sin/cos of this timer — used by D2's
            output stage for tension. Pass None for steady cam.

    Returns:
        (fcx, fcy) — the final render camera position with shake and kick
        applied ON TOP (not smoothed by the follow lerp).

    Design note on shake / kick:
        Earlier versions baked ``sx``, ``sy``, ``cam_kick_x``, ``cam_kick_y``
        into ``desired_x`` / ``desired_y`` and then output ``STATE.camera_x``
        toward desired at 0.05/frame. That smoothed the shake and kick into
        a barely-visible drift: a ``shake(10)`` states that produced only
        ~3 pixels of visible jitter per frame. The fix is to compute the
        player-follow position via lerp (so the camera still tracks the
        player smoothly) and then add shake + kick OFFSET to the returned
        render position. This restores the snappy, "punchy" feel of D1's
        pre-refactor shake.

    Side effects:
        Mutates STATE.camera_x, STATE.camera_y (player-follow position only),
        STATE.screen_shake (decays toward 0), STATE.cam_kick_x/y (decays
        toward 0).
    """
    tx, ty = target

    # -- Screen shake update (non-clamp semantics, never direct assignment) --
    # Computed per frame as a fresh random offset, then applied directly to
    # the returned render position below.
    sx = sy = 0
    if getattr(STATE, "reduced_motion", False):
        STATE.screen_shake = 0
    elif STATE.screen_shake > 0:
        sx = random.randint(-STATE.screen_shake, STATE.screen_shake)
        sy = random.randint(-STATE.screen_shake, STATE.screen_shake)
        STATE.screen_shake = max(0, STATE.screen_shake - 1)

    # -- Recoil kick decay (set by weapons.handle_shoot on every shot) --
    # The kick value itself decays exponentially; the current value is added
    # directly to the returned render position below.
    STATE.cam_kick_x *= 0.80
    STATE.cam_kick_y *= 0.80

    # -- Desired position is the PLAYER-FOLLOW target only --
    # Shake and kick are NOT baked in here — that was the bug that made
    # it recoil feel mushy. They're applied directly to the render return.
    desired_x = tx - WIDTH // 2
    desired_y = ty - HEIGHT // 2

    # -- Map-bounds clamp on the "desired" position (D2's approach) --
    if map_w is not None and map_h is not None:
        max_x = max(0, map_w * TILE_SIZE - WIDTH)
        max_y = max(0, map_h * TILE_SIZE - HEIGHT)
        desired_x = max(0, min(max_x, desired_x))
        desired_y = max(0, min(max_y, desired_y))

    # -- First-frame snap: avoids the camera sweeping in from (0,0) --
    if not getattr(STATE, "camera_initialized", False):
        STATE.camera_x = desired_x
        STATE.camera_y = desired_y
        STATE.camera_initialized = True
    else:
        # -- Lerp toward desired position (player follow only) --
        dx = desired_x - STATE.camera_x
        dy = desired_y - STATE.camera_y
        if factor_fn is not None:
            dist = math.hypot(dx, dy)
            if dist < 0.01:
                STATE.camera_x = desired_x
                STATE.camera_y = desired_y
            else:
                factor = factor_fn(dx, dy, dist)
                STATE.camera_x += dx * factor
                STATE.camera_y += dy * factor
        else:
            ease = 0.10 if focus_factor is None else focus_factor
            STATE.camera_x += dx * ease
            STATE.camera_y += dy * ease

    # -- Apply shake + kick OFFSET to the render return --
    # This is the key fix: by adding them AFTER the lerp, we avoid the
    # smoothing that was making recoil feel mushy. The render position can
    # go briefly out the from shake amplitude each frame, and the kick pushes
    # the camera in the opposite direction of the shot immediately.
    render_x = STATE.camera_x + STATE.cam_kick_x + sx
    render_y = STATE.camera_y + STATE.cam_kick_y + sy

    # -- Handheld jitter (optional, for high-tension stages) --
    if handheld_jitter_time is not None and handheld_jitter_time > 0:
        if not getattr(STATE, "reduced_motion", False):
            render_x += math.sin(handheld_jitter_time * 0.011) * 0.6
            render_y += math.cos(handheld_jitter_time * 0.013) * 0.6

    return render_x, render_y

def reset_camera_state():
    """Reset the camera_initialized flag so the next update_camera snaps."""
    STATE.camera_initialized = False
    STATE.cam_kick_x = 0.0
    STATE.cam_kick_y = 0.0

def collide_bullet_walls(b, solid_walls):
    """Swept bullet-vs-wall collision.

    Uses ``Rect.clipline(prev_pos, cur_pos)`` so that fast bullets (20 px/frame
    and up) cannot tunnel through thin walls. Returns True if the bullet hit
    a wall; the caller is responsible for spawning impact particles / sounds.

    Important: a bullet that STARTS inside a wall (e.g. an enemy whose gun
    position is inside a tiny wall) and moves outward is NOT counted as a
    wall hit. Only bullets that cross from outside a wall into it register,
    without this exception, enemies standing behind cover fire bullets that
    are absorbed instantly and never reach the player.
    """
    for wall in solid_walls:
        if wall.clipline((b["prev_x"], b["prev_y"]), (b["x"], b["y"])):
            # Skip if the bullet was already inside this wall on the previous
            # frame — it spawned there (enemy gun pre inside cover) and is
            # moving outward, not entering the wall from outside.
            if not wall.collidepoint(b["prev_x"], b["prev_y"]):
                return True
    return False

def draw_bullets(surface, bullets, cam_x, cam_y, with_glow=True):
    """Render bullets with optional glow halo, motion trail, and their core.

    The glow halo is the soft yellow dome surface from config.bullet_glow.
    Both dinosaurs now render it; previously D1 skipped it, making D1 bullets
    look thin crisp lines while D2's looked like glowing tracers.
    """
    for b in bullets:
        bx = int(b["x"] - cam_x)
        by = int(b["y"] - cam_y)
        if with_glow:
            surface.blit(bullet_glow, (bx - 8, by - 8))
        pygame.draw.circle(surface, C_BULLET, (bx, by), 3)
        trail_x = b.get("prev_x", b["x"] - b["dx"]) - cam_x
        trail_y = b.get("prev_y", b["y"] - b["dy"]) - cam_y
        pygame.draw.line(
            surface, (255, 200, 100),
            (bx, by), (trail_x, trail_y), 2,
        )
```